

Paso de Información entre Componentes (Props y Eventos)

Objetivo de la Guía

Comprender cómo compartir información entre componentes en React utilizando props y cómo comunicar eventos desde componentes hijos hacia componentes padres mediante funciones.

Actividad de Reflexión Inicial

En una aplicación real, los componentes no trabajan de forma aislada. Un botón puede necesitar enviar información a otro componente, o un formulario debe comunicar datos al sistema principal.

Actividad

Responda:

1. ¿Por qué cree que los componentes necesitan comunicarse entre sí?
R/: Pienso que los componentes necesitan comunicarse para compartir información y trabajar juntos dentro de una aplicación.
2. ¿Qué problemas podrían surgir si los componentes no comparten información?
R/: La aplicación podría funcionar de manera incorrecta, mostrar información desactualizada o no responder correctamente a las acciones del usuario.
3. ¿Qué tipo de datos cree que un componente podría enviar a otro?
R/: Puede enviar textos también números, objetos, listas, información de usuarios, productos y funciones para realizar determinadas acciones.
4. ¿Por qué es importante mantener organizada la comunicación entre componentes?
R/: Porque este facilita entender, mantener y modificar el código. Además, evita errores y hace que los componentes puedan reutilizarse fácilmente.
5. ¿Cómo cree que se puede enviar información desde un componente pequeño a uno principal?
1. R/: Se puede hacer mediante una función que el componente padre envía al hijo como una prop. El hijo ejecuta esa función cuando ocurre una acción y así puede enviar información al padre.

Actividad de Contextualización

En React, la comunicación entre componentes es fundamental para construir aplicaciones dinámicas. Ingrese a <https://share.gemini.google/HrOrbhnl29A5>, desarrolle la actividad y basada en las preguntas que se presentan en la actividad responda:

1. ¿Qué son las props en React?
R/: Las props son propiedades que permiten enviar información desde un componente padre hacia un componente hijo.
2. ¿Para qué se utilizan?
R/: Estos se utilizan para enviar datos, personalizar componentes y hacer que un mismo componente pueda volver a utilizarse con diferentes valores.

3. ¿Cómo se comunican los componentes entre sí?
R/: Principalmente mediante props. El padre envía información al hijo y, para que el hijo pueda comunicar una acción al padre, se puede enviar una función como prop.
4. ¿Qué ventaja tiene reutilizar componentes con props?
R/: Permite escribir un componente una sola vez y utilizarlo en diferentes lugares cambiando sus datos mediante props.

Apropiación del Conocimiento

¿Qué son las Props?

Las props (propiedades) son datos o parámetros que un componente padre le envía a un componente hijo para configurar o personalizar su contenido.

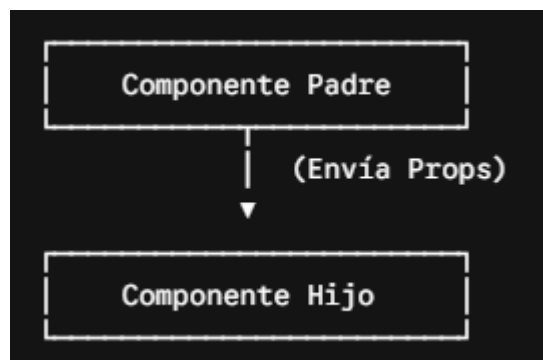
Analogía: Piense en un componente como un molde de galletas. El molde siempre hace la misma forma, pero las props son la chispas de chocolate, la vainilla o el glaseado que le agregas a cada una para que sean diferentes.

¿Para qué sirven?

- Personalizar contenido: Un mismo botón puede decir "Aceptar", "Cancelar" o "Eliminar".
- Reutilizar componentes: Escribe el código una sola vez y lo adapta según el lugar donde lo uses.
- Compartir información: Transmitir datos de la lógica principal hacia las vistas secundarias.

Flujo de Datos Unidireccional

En React, la información viaja en una sola dirección: de Arriba hacia Abajo (Padre → Hijo).



Los componentes hijos no pueden modificar directamente las props que reciben (son de solo lectura).

Ejemplos Prácticos de Props

1. Ejemplo Básico (Un solo dato)

Hijo (Mensaje.tsx):

```
interface MensajeProps {
  texto: string;
}

export function Mensaje({ texto }: MensajeProps) {
  return <h3>{texto}</h3>;
}
```

Padre (App.tsx):

```
import { Mensaje } from './Mensaje';

export function App() {
  return (
    <div>
      {/* Pasamos la prop "texto" al hijo */}
      <Mensaje texto="¡Bienvenido a la plataforma!" />
      <Mensaje texto="Tienes 3 notificaciones pendientes." />
    </div>
  );
}
```

2. Props con Múltiples Datos

Hijo (TarjetaUsuario.tsx):

```
interface TarjetaProps {
  nombre: string;
  rol: string;
  edad: number;
}

export function TarjetaUsuario({ nombre, rol, edad }: TarjetaProps) {
  return (
    <div className="tarjeta">
      <h2>{nombre}</h2>
      <p>Rol: {rol}</p>
      <p>Edad: {edad} años</p>
    </div>
  );
}
```

Padre (App.tsx):

```
import { Mensaje } from './Mensaje';
import { TarjetaUsuario } from './TarjetaUsuario';
export function App() {
  return (
    <div>
      {/* Pasamos la prop "texto" al hijo */}
      <Mensaje texto="¡Bienvenido a la plataforma!" />
      <Mensaje texto="Tienes 3 notificaciones pendientes." />
      <TarjetaUsuario
        nombre="Freddy Ardila"
        rol="Instructor"
        edad={30}
      />
    </div>
  );
}
```

¿Qué son los Eventos?

Los eventos son acciones que realiza el usuario en la interfaz y que el sistema debe escuchar para reaccionar a ellas.

Ejemplos comunes:

- **onClick:** El usuario hace clic en un botón.
- **onChange:** El usuario escribe en un campo de texto.
- **onSubmit:** El usuario envía un formulario.

Comunicación Hijo → Padre (Pasar funciones como Props)

Como la información en React solo baja (Padre → Hijo), ¿cómo le avisa un hijo al padre que ocurrió algo?

En React no existe \$emit (como en Vue). La solución es que el Padre le entrega una función al Hijo como si fuera una prop. Cuando ocurre el evento, el Hijo ejecuta esa función.

Ejemplo Simple (Notificar una acción)

Hijo (BotonAccion.tsx):

```
interface BotonProps {
  onAvisarPadre: () => void;
}

export function BotonAccion({ onAvisarPadre }: BotonProps) {
  return (
    <button onClick={onAvisarPadre}>
      Presiona el botón
    </button>
  );
}
```

Padre (App.tsx):

```
import { BotonAccion } from './BotonAccion';
export function App() {
  const manejarClikeo = () => {
    alert("¡El componente hijo me avisó que hicieron clic!");
  };
  return <BotonAccion onAvisarPadre={manejarClikeo} />;
}
```

2. Ejemplo Enviando Datos del Hijo al Padre

Hijo (Buscador.tsx):

```
interface BuscadorProps {
  onBuscar: (termino: string) => void;
}

export function Buscador({ onBuscar }: BuscadorProps) {
  return (
    <button onClick={() => onBuscar("React Native")}>
      Buscar "React Native"
    </button>
  );
}
```

Padre (App.tsx):

```
import { BotonAccion } from './BotonAccion';
import { Buscador } from './Buscador';
export function App() {
  // 1. Declaramos todas las funciones controladoras
  const manejarClikeo = () => {
    alert("¡El componente hijo me avisó que hicieron clic!");
  };
  const recibirBusqueda = (texto: string) => {
    // Usamos backticks `` para interpolar variables con ${}
    alert(`Buscando en la base de datos: ${texto}`);
  };
  // 2. Retornamos UN SOLO bloque JSX que agrupa a ambos componentes
  return (
    <main style={{ padding: '20px', display: 'flex', flexDirection: 'column', gap: '15px' }}>
      <BotonAccion onAvisarPadre={manejarClikeo} />
      <Buscador onBuscar={recibirBusqueda} />
    </main>
  );
}
```

Buenas Prácticas

- **Tipar correctamente las Props:** Usar siempre interface o type en TypeScript para definir qué datos se reciben.
- **Nombres claros:** Iniciar las funciones con on cuando se pasan como props (ej: onEliminar, onGuardar) y con handle al definir las en el padre (ej: handleEliminar).
- **Evitar Prop Drilling excesivo:** No pases props a través de 5 o 6 niveles de componentes si no las necesitan.

- **Mantener la estructura limpia:** El componente hijo solo debe mostrar la vista y emitir eventos; el padre debe controlar la lógica y los datos.

Resumen

- **Props:** Envían datos del componente Padre al Hijo.
- **Funciones como Props:** Permiten que el Hijo responda o envíe datos al Padre.
- **Flujo Unidireccional:** Los datos en React siempre fluyen de arriba hacia abajo.

Actividad de Apropiación del Conocimiento

Actividad 1 – Tabla de conceptos

Complete:

Concepto	Definición	Ejemplo de código sencillo
Props	Dato que un componente padre envía a un componente hijo.	<code><Producto nombre="Laptop" /></code>
Evento	Acción realizada por el usuario que genera una respuesta en la app.	<code><button onClick={handleClick}>Clic</button></code>
Componente padre	Componente que contiene y utiliza a otro componente y pueda enviarle props.	<code><Producto nombre="Mouse" /></code>
Componente hijo	Componente que recibe información del padre mediante props.	<code>function Producto({ nombre }) { return <p>{nombre}</p> }</code>

Actividad 2 – Desarrollo práctico

Cree:

- un componente Producto que reciba nombre y precio
- un componente App que envíe los datos

Pasos

1. Cree el proyecto: `npm create vite@latest padre-hijo -- --template react-ts`
2. # Entrar a la carpeta del proyecto
`cd padre-hijo`
3. # Instalar dependencias
`npm install`
4. Una vez abierto en VS Code, verá esta estructura:

```

├─ src/
│   └─ App.tsx          # Componente principal
│   └─ main.tsx         # Punto de entrada
│   └─ ...
├─ package.json
└─ ...

```

5. Cree la carpeta components y dentro de la carpeta, genere el siguiente archivo llamado: Producto.tsx

```

function Producto(props: { nombre: string; precio: number; image: string }) {
  return (
    <div>
      <img src={props.image}/>
      <h3>{props.nombre}</h3>
      <p>Precio: ${props.precio}</p>
    </div>
  );
}

export default Producto;

```

```

interface ProductoProps {
  nombre: string;
  precio: number;
}

```

```

function Producto({ nombre, precio }: ProductoProps) {
  return (
    <div>
      <h2>{nombre}</h2>
      <p>Precio: ${precio.toLocaleString("es-CO")}</p>
    </div>
  );
}

```

```

export default Producto;

```

6. En el componente App.tsx, reemplace el código existente, por este:

```
import Producto from "../components/Producto";
//import BotonMensaje from "../components/BotonMensaje";
function App() {

  return (
    <div style={{ textAlign: "center" }}>
      <h1>La tienda de Freddy</h1>
      <Producto image="image1.jpg" nombre="Portatil" precio={4000000} />
      <Producto image="image2.jpg" nombre="Mouse" precio={45000} />
      <Producto image="image3.jpg" nombre="Teclado" precio={75000} />
    </div>
  );
}
export default App;
```

```
import Producto from "../components/producto";
```

```
function App() {
```

```
  return (
```

```
    <div>
```

```
      <h1>Lista de Productos</h1>
```

```
      <Producto nombre="Laptop" precio={2500000} />
```

```
      <Producto nombre="Mouse" precio={45000} />
```

```
      <Producto nombre="Teclado" precio={80000} />
```

```
      <Producto nombre="Monitor" precio={650000} />
```

```
      <Producto nombre="Audífonos" precio={120000} />
```

```
      <Producto nombre="Memoria USB" precio={35000} />
```

```
      <Producto nombre="Cámara Web" precio={150000} />
```

```
      <Producto nombre="Parlantes" precio={100000} />
```

```
      <Producto nombre="Disco SSD" precio={280000} />
```

```
      <Producto nombre="Impresora" precio={550000} />
```

```
    </div>
```

```
  );
```

```
}
```

```
export default App;
```


7. Iguale Main.tsx a este contenido:

```
import React from 'react'
import ReactDOM from 'react-dom/client'
import App from './App'

ReactDOM.createRoot(document.getElementById('root')!).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>,
)
```

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import App from "./App";

createRoot(document.getElementById("root")!).render(
  <StrictMode>
    <App />
  </StrictMode>
);
```

8. Por último, ejecute el proyecto con: *npm run dev*

9. Modifique el documento para que tenga 10 precios y 10 productos y anexe el código al documento que va a entregar.

10. Opción de mejora (agregue imágenes)

Actividad 3 – Comunicación

Cree:

- Un botón en un componente hijo
- Al hacer clic, debe mostrar un mensaje en el componente padre

Pasos

1. Basado en el proyecto anterior, genere un nuevo componente llamado BotonMensaje.tsx y agregue este código:

```
function BotonMensaje({ onClick }: { onClick: () => void }) {
  return <button onClick={onClick}>Enviar mensaje</button>;
}

export default BotonMensaje;
```

```
interface BotonMensajeProps {
  mensaje: string;
  onMensaje: (mensaje: string) => void;
}
```

```
function BotonMensaje({ mensaje, onMensaje }: BotonMensajeProps) {
  return (
```

```

        <button onClick={() => onMensaje(mensaje)}>
            Mostrar mensaje
        </button>
    );
}

```

export default BotonMensaje;

2. Ahora modifique el archivo App.tsx para que quede así:

```

import Producto from "../components/Producto";
import BotonMensaje from "../components/BotonMensaje";
function App() {
    function mostrarMensaje() {
        alert("¡Botón clickeado!");
    }
    return (
        <div style={{ textAlign: "center" }}>
            <h1>Mi Tienda</h1>
            <Producto nombre="Laptop" precio={800} />
            <Producto nombre="Mouse" precio={20} />
            <Producto nombre="Teclado" precio={50} />
            <BotonMensaje onClick={mostrarMensaje} />
        </div>
    );
}
export default App;

```

import BotonMensaje from "../components/BotonMensaje";

```

function App() {
    const handleMensaje = (mensaje: string) => {
        alert(mensaje);
    };

    return (
        <div>
            <h1>Comunicación entre componentes</h1>

            <BotonMensaje
                mensaje="Hola, seleccionaste el botón 1"
                onMensaje={handleMensaje}
            />

            <BotonMensaje
                mensaje="Hola, seleccionaste el botón 2"
                onMensaje={handleMensaje}
            />
        </div>
    );
}

```

```

    <BotonMensaje
      mensaje="Hola, seleccionaste el botón 3"
      onMensaje={handleMensaje}
    />
  </div>
);
}

```

export default App;

3. Ponga al menos 3 botones

Transferencia del Conocimiento

Actividad 1 – Sistema empresarial

Diseñe un módulo con:

- Componente padre (App)
- 3 componentes hijos
- Cada hijo debe recibir información diferente mediante props.

USUARIOS.TSX

```

interface UsuariosProps {
  nombre: string;
  estado: string;
}

function Usuarios({ nombre, estado }: UsuariosProps) {
  return (
    <div>
      <h2>Usuarios</h2>
      <p>Nombre: {nombre}</p>
      <p>Estado: {estado}</p>
    </div>
  );
}

export default Usuarios;

```

PRODUCTOS.TSX

```
interface ProductosProps {  
  nombre: string;  
  cantidad: number;  
}
```

```
function Productos({ nombre, cantidad }: ProductosProps) {  
  return (  
    <div>  
      <h2>Productos</h2>  
      <p>Producto: {nombre}</p>  
      <p>Cantidad: {cantidad}</p>  
    </div>  
  );  
}
```

```
export default Productos;
```

REPORTES.TSX

```
interface ReportesProps {  
  tipo: string;  
  periodo: string;  
}
```

```
function Reportes({ tipo, periodo }: ReportesProps) {  
  return (  
    <div>  
      <h2>Reportes</h2>  
      <p>Tipo de reporte: {tipo}</p>  
      <p>Periodo: {periodo}</p>  
    </div>  
  );  
}
```

```
export default Reportes;
```

APP.TSX

```
import Usuarios from "../components/Usuarios";
import Productos from "../components/Productos";
import Reportes from "../components/Reportes";

function App() {
  return (
    <div style={{ textAlign: "center" }}>
      <h1>Rastreo Ambiental</h1>

      <h2>Sistema Empresarial</h2>

      <Usuarios
        nombre="Administrador"
        estado="Activo"
      />

      <Productos
        nombre="Punto de Reciclaje"
        cantidad={10}
      />

      <Reportes
        tipo="Reporte Ambiental"
        periodo="Mensual"
      />
    </div>
  );
}

export default App;
```

Actividad 2 – Creación de un formulario

Construir un formulario en React con los campos nombre, teléfono, correo y mensaje, usando props para comunicar datos entre un componente padre y un hijo.

Instrucciones:

- El padre (App.tsx) debe guardar el estado de los cuatro campos y pasar los valores y funciones al hijo como props.
- El hijo (Formulario.tsx) debe mostrar los inputs y el botón de envío, usando las props recibidas.
- Al enviar, se deben mostrar los datos ingresados en un alert.

Formulario.tsx

```
import React from 'react';

interface FormularioProps {
  datos: {
    nombre: string;
    telefono: string;
    correo: string;
    mensaje: string;
  };
  onChange: (e: React.ChangeEvent<HTMLInputElement | HTMLTextAreaElement>) => void;
  onSubmit: (e: React.FormEvent<HTMLFormElement>) => void;
}

const Formulario: React.FC<FormularioProps> = ({ datos, onChange, onSubmit }) => {
```

FORMULARIO.TSX

```
interface FormularioProps {
```

```
  nombre: string;
  telefono: string;
  correo: string;
  mensaje: string;
```

```
  onNombre: (valor: string) => void;
  onTelefono: (valor: string) => void;
  onCorreo: (valor: string) => void;
  onMensaje: (valor: string) => void;
```

```
  onEnviar: () => void;
}
```

```
function Formulario({
```

```
  nombre,
  telefono,
  correo,
  mensaje,
  onNombre,
  onTelefono,
  onCorreo,
  onMensaje,
  onEnviar
```

```
}: FormularioProps) {
```

```
  return (
    <div>
      <h2>Formulario de Contacto</h2>

      <input
        type="text"
```

```

        placeholder="Nombre"
        value={nombre}
        onChange={(e) => onNombre(e.target.value)}
    />

    <br /><br />

    <input
        type="text"
        placeholder="Teléfono"
        value={telefono}
        onChange={(e) => onTelefono(e.target.value)}
    />

    <br /><br />

    <input
        type="email"
        placeholder="Correo"
        value={correo}
        onChange={(e) => onCorreo(e.target.value)}
    />

    <br /><br />

    <textarea
        placeholder="Mensaje"
        value={mensaje}
        onChange={(e) => onMensaje(e.target.value)}
    />

    <br /><br />

    <button onClick={onEnviar}>
        Enviar
    </button>
</div>
);
}

export default Formulario;

```

App.tsx

```
import React from "react";
import Formulario from "../components/Formulario";

function App() {
  const [formData, setFormData] = React.useState({
    nombre: "",
    telefono: "",
    correo: "",
    mensaje: ""
  });

  const manejarCambio = (e) => {
    setFormData({
      ...formData,
      [e.target.name]: e.target.value
    });
  };

  const manejarEnvio = (e) => {
    e.preventDefault();
    // Solo mostrar los datos en un alert
    alert(
      `Nombre: ${formData.nombre}\nTeléfono: ${formData.telefono}\nCorreo: ${formData.correo}\nMensaje: ${formData.mensaje}`
    );
  };

  return (
    <Formulario
      datos={formData}
      onChange={manejarCambio}
      onSubmit={manejarEnvio}
    />
  );
}
export default App;
```

APP.TSX

```
import Formulario from "../components/Formulario";
```

```
function App() {
  let nombre = "";
  let telefono = "";
  let correo = "";
  let mensaje = "";
```

```
  const handleNombre = (valor: string) => {
    nombre = valor;
  };
```

```
  const handleTelefono = (valor: string) => {
    telefono = valor;
  };
```

```
  const handleCorreo = (valor: string) => {
    correo = valor;
  };
```

```
  const handleMensaje = (valor: string) => {
```



```

    mensaje = valor;
  };

  const handleEnviar = () => {
    alert(
      "Nombre: " + nombre +
      "\nTeléfono: " + telefono +
      "\nCorreo: " + correo +
      "\nMensaje: " + mensaje
    );
  };

  return (
    <div style={{ textAlign: "center" }}>
      <h1>Rastreo Ambiental</h1>

      <Formulario
        nombre={nombre}
        telefono={telefono}
        correo={correo}
        mensaje={mensaje}
        onNombre={handleNombre}
        onTelefono={handleTelefono}
        onCorreo={handleCorreo}
        onMensaje={handleMensaje}
        onEnviar={handleEnviar}
      />
    </div>
  );
}

export default App;

```

Actividad 3 – Proyecto formativo

Además de los archivos Usuarios.tsx y Productos.tsx desarrollados anteriormente, usted debe crear 3 nuevos componentes de interfaz que representen módulos adicionales de su proyecto productivo. Entre ellos deben estar obligatoriamente Login.tsx y Registro.tsx, y un tercero a elección (ejemplo: Ventas.tsx, Inventario.tsx, Configuración.tsx, Reportes.tsx, etc.).

Para cada una de las 5 interfaces (Usuarios, Productos, Login, Registro y el módulo adicional) debe implementar lo siguiente:

- Componente hijo reutilizable: Cree un componente (ej: CardAccion.tsx) que sea llamado dentro de estas interfaces.
- Paso de props (Padre → Hijo): El padre debe enviar información específica (nombres, estados o textos) para que el hijo los renderice.
- Pasos de Hijo → Padre: recibir información por medio de funciones

- Acción en el padre: Al recibir la señal del hijo, el padre debe mostrar un alert() o console.log() indicando qué acción se realizó y en qué módulo.
- Login y Registro: En estos dos módulos, al enviar el formulario se debe mostrar la información ingresada en un alert() (ejemplo: usuario, contraseña, correo, etc.).
- No olvide modificar el archivo App.tsx, para los estilos.
- Tome cómo base este archivo y añada los botones para ver todos los componentes desarrollados (los dos de la sesión anterior y los 5 nuevos)

```
import { useState } from 'react';
import Usuarios from './components/Usuarios';
import Productos from './components/Productos';

function App() {
  // Definimos qué página mostrar: 'usuarios' o 'productos'
  const [pagina, setPagina] = useState<string>('usuarios');

  return (
    <div style={{ fontFamily: 'sans-serif', padding: '20px' }}>
      <nav style={{ marginBottom: '20px' }}>
        <button onClick={() => setPagina('usuarios')}>Ver Usuarios</button>
        <button onClick={() => setPagina('productos')} style={{ marginLeft: '10px' }}>Ver Productos</button>
      </nav>

      <hr />

      { /* Lógica para mostrar un componente u otro */ }
      <main>
        {pagina === 'usuarios' && <Usuarios />}
        {pagina === 'productos' && <Productos />}
      </main>
    </div>
  );
}

export default App;
```

CardAction.tsx

```
interface CardAccionProps {
  titulo: string;
  descripcion: string;
  textoBoton: string;
  onAccion: () => void;
}
```

```
function CardAccion({
  titulo,
  descripcion,
  textoBoton,
  onAccion
}: CardAccionProps) {
  return (
    <div className="card-accion">
      <h3>{titulo}</h3>
      <p>{descripcion}</p>
```

```

        <button onClick={onAccion}>
          {textoBoton}
        </button>
      </div>
    );
  }

```

```
export default CardAccion;
```

USUARIOS.TSX

```
import CardAccion from "../components/CardAccion";
```

```

interface UsuariosProps {
  nombreModulo: string;
}

```

```
function Usuarios({ nombreModulo }: UsuariosProps) {
```

```

  const manejarAccion = () => {
    alert(`Se realizó una acción en el módulo ${nombreModulo}`);
  };

```

```

  return (
    <div className="modulo">
      <h2>Gestión de Usuarios</h2>

```

```

      <CardAccion
        titulo="Usuarios registrados"
        descripcion="Administra los usuarios del sistema de Rastreo Ambiental."
        textoBoton="Gestionar usuarios"
        onAccion={manejarAccion}
      />
    </div>

```

```

  );
}

```

```
export default Usuarios;
```

PRODUCTOS.TSX

```
import CardAccion from "../components/CardAccion";

interface ProductosProps {
  nombreModulo: string;
}

function Productos({ nombreModulo }: ProductosProps) {

  const manejarAccion = () => {
    alert(`Se realizó una acción en el módulo ${nombreModulo}`);
  };

  return (
    <div className="modulo">
      <h2>Gestión de Productos</h2>

      <CardAccion
        titulo="Productos ambientales"
        descripcion="Administra materiales y elementos relacionados con el reciclaje."
        textoBoton="Ver productos"
        onAccion={manejarAccion}
      />
    </div>
  );
}

export default Productos;
```

LOGIN.TSX

```
import CardAccion from "../components/CardAccion";

interface LoginProps {
  nombreModulo: string;
}

function Login({ nombreModulo }: LoginProps) {

  const manejarLogin = (
    event: React.FormEvent<HTMLFormElement>
  ) => {
    event.preventDefault();

    const formulario = event.currentTarget;
```

```

const usuario = (
  formulario.elements.namedItem("usuario") as HTMLInputElement
).value;

const contraseña = (
  formulario.elements.namedItem("contraseña") as HTMLInputElement
).value;

alert(
  `Módulo: ${nombreModulo}\nUsuario: ${usuario}\nContraseña: ${contraseña}`
);
};

const manejarAccion = () => {
  console.log(`Acción realizada en ${nombreModulo}`);
};

return (
  <div className="modulo">
    <h2>Inicio de Sesión</h2>

    <form onSubmit={manejarLogin} className="formulario">

      <label>Usuario</label>
      <input
        type="text"
        name="usuario"
        placeholder="Ingrese su usuario"
        required
      />

      <label>Contraseña</label>
      <input
        type="password"
        name="contraseña"
        placeholder="Ingrese su contraseña"
        required
      />

      <button type="submit">
        Iniciar sesión
      </button>

    </form>

    <CardAccion
      titulo="Acceso al sistema"
      descripcion="Ingresa al sistema de Rastreo Ambiental."
    >

```

```

        textoBoton="Acción del módulo"
        onAccion={manejarAccion}
      />
    </div>
  );
}

```

```
export default Login;
```

REGISTRO.TSX

```
import CardAccion from "../components/CardAccion";
```

```

interface RegistroProps {
  nombreModulo: string;
}

```

```
function Registro({ nombreModulo }: RegistroProps) {
```

```

  const manejarRegistro = (
    event: React.FormEvent<HTMLFormElement>
  ) => {
    event.preventDefault();

```

```
    const formulario = event.currentTarget;
```

```

    const nombre = (
      formulario.elements.namedItem("nombre") as HTMLInputElement
    ).value;

```

```

    const correo = (
      formulario.elements.namedItem("correo") as HTMLInputElement
    ).value;

```

```

    const contraseña = (
      formulario.elements.namedItem("contraseña") as HTMLInputElement
    ).value;

```

```

    alert(
      `Módulo: ${nombreModulo}\n` +
      `Nombre: ${nombre}\n` +
      `Correo: ${correo}\n` +
      `Contraseña: ${contraseña}`
    );
  };

```

```
const manejarAccion = () => {
```

```

    console.log(`Acción realizada en ${nombreModulo}`);
};

return (
  <div className="modulo">
    <h2>Registro de Usuario</h2>

    <form onSubmit={manejarRegistro} className="formulario">

      <label>Nombre completo</label>
      <input
        type="text"
        name="nombre"
        placeholder="Ingrese su nombre"
        required
      />

      <label>Correo electrónico</label>
      <input
        type="email"
        name="correo"
        placeholder="Ingrese su correo"
        required
      />

      <label>Contraseña</label>
      <input
        type="password"
        name="contraseña"
        placeholder="Ingrese una contraseña"
        required
      />

      <button type="submit">
        Registrarse
      </button>

    </form>

    <CardAccion
      titulo="Nuevo usuario"
      descripcion="Crea una cuenta para acceder al sistema."
      textoBoton="Acción del registro"
      onAccion={manejarAccion}
    />
  </div>
);
}

```

```
export default Registro;
```

REPORTES.TSX

```
import CardAccion from "../components/CardAccion";
```

```
interface ReportesProps {  
  nombreModulo: string;  
}
```

```
function Reportes({ nombreModulo }: ReportesProps) {
```

```
  const manejarAccion = () => {  
    alert(`Se generó una acción en el módulo ${nombreModulo}`);  
  };
```

```
  return (  
    <div className="modulo">  
      <h2>Reportes Ambientales</h2>  
  
      <CardAccion  
        titulo="Reporte ambiental"  
        descripcion="Consulta información sobre residuos y puntos de reciclaje."  
        textoBoton="Generar reporte"  
        onAccion={manejarAccion}  
      />  
    </div>  
  );  
}
```

```
export default Reportes;
```

APP.TSX

```
import Usuarios from "../pages/Usuarios";  
import Productos from "../pages/Productos";  
import Login from "../pages/Login";  
import Registro from "../pages/Registro";  
import Reportes from "../pages/Reportes";
```

```
function App() {
```

```
  return (  
    <div className="app">
```



```

<header>
  <h1>Rastreo Ambiental</h1>
  <p>Sistema de gestión y seguimiento ambiental</p>
</header>

<main>

  <section>
    <h2>Componentes del sistema</h2>

    <Usuarios nombreModulo="Usuarios" />

    <Productos nombreModulo="Productos" />

    <Login nombreModulo="Login" />

    <Registro nombreModulo="Registro" />

    <Reportes nombreModulo="Reportes" />

  </section>

</main>

<footer>
  <p>Rastreo Ambiental © 2026</p>
</footer>

</div>
);
}

export default App;

```

APP.CSS

```

* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}

body {
  font-family: Arial, sans-serif;
  background-color: #f2f5f3;
}

```

```
.app {
  min-height: 100vh;
}

header {
  background-color: #176b3a;
  color: white;
  text-align: center;
  padding: 30px;
}

header h1 {
  margin-bottom: 10px;
}

main {
  width: 90%;
  max-width: 1000px;
  margin: 30px auto;
}

main h2 {
  text-align: center;
  margin-bottom: 25px;
}

.modulo {
  background-color: white;
  padding: 25px;
  margin-bottom: 25px;
  border-radius: 10px;
  box-shadow: 0 3px 10px rgba(0, 0, 0, 0.1);
}

.modulo h2 {
  color: #176b3a;
}

.card-accion {
  border: 1px solid #ddd;
  padding: 20px;
  border-radius: 8px;
  text-align: center;
}

.card-accion h3 {
  margin-bottom: 10px;
}
```

```
.card-accion p {  
  margin-bottom: 15px;  
  color: #555;  
}
```

```
button {  
  background-color: #176b3a;  
  color: white;  
  border: none;  
  padding: 10px 20px;  
  border-radius: 6px;  
  cursor: pointer;  
}
```

```
button:hover {  
  background-color: #0f4d29;  
}
```

```
.formulario {  
  display: flex;  
  flex-direction: column;  
  gap: 10px;  
  margin-bottom: 25px;  
}
```

```
.formulario input {  
  padding: 10px;  
  border: 1px solid #ccc;  
  border-radius: 5px;  
}
```

```
footer {  
  background-color: #176b3a;  
  color: white;  
  text-align: center;  
  padding: 20px;  
}
```

- Cargue el repositorio a GitHub basado en el siguiente vídeo: [GitHub](#) y comparta el link del github por proyecto.

Restricciones Técnicas:

Sin Hooks: Está prohibido el uso de useState o cualquier otro hook; la comunicación debe ser directa por funciones de callback.

TypeScript: Es obligatorio el uso de interface para definir todas las propiedades de los componentes.

REPOSITORIO: <https://github.com/rincondilan278-ship-it/proyecto-sena-rastreo.git>